UNIVERSIDAD CATÓLICA DE TEMUCO
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA
FACULTAD DE INGENIERÍA

PLATAFORMA WEB PARA EL REGISTRO, MONITOREO Y GESTIÓN DE DATOS DE SENSORES

Recepción, normalización y supervisión de tramas de telemetría, validada en el Campus Dr. Luis Rivas del Canto de la Universidad Católica de Temuco

por

VICENTE RICARDO RAMÍREZ OÑATE

Trabajo de Título presentado a la
Facultad de Ingeniería de la Universidad Católica de Temuco
Para Optar al Título de Ingeniero Civil Informático.

Curso INFO-1197 · Profesor guía: Mario Villanueva
Etapa 1: Definición del Problema, Objetivos y Análisis de Requerimientos
Entrega: 4 de septiembre de 2026

- Temuco, septiembre 2026 -

Índice general

Índice general I

Índice de figuras IV

Índice de cuadros V

Resumen VI

1 Introducción **1**
1.1. Problema 1
1.2. Estado del arte 3
1.3. Objetivos 4
1.3.1. Objetivo general 4
1.3.2. Objetivos específicos 4
1.4. Autoría y licenciamiento 6
1.5. Estructura del documento 6

2 Antecedentes **7**
2.1. Los protocolos se apilan, no compiten 7
2.2. Mensajería MQTT 8
2.3. El formato de trama: SenML 10
2.4. La rama de radio: LoRa, LoRaWAN y ChirpStack 10
2.5. Persistencia de series de tiempo 11
2.6. Seguridad del transporte 11

3 Análisis de requerimientos **12**
3.1. Metodología del levantamiento 12
3.2. Síntesis del catálogo 12

3.3. Evaluación del mecanismo de transporte 13
3.4. Evaluación del formato de trama 14
3.5. Arquitectura de recepción y frontera del trabajo 15
3.6. Evaluación de infraestructura 17

4 Metodología y planificación 18
4.1. Marco metodológico 18
4.2. Calendario oficial 19
4.3. Backlog y verificación de viabilidad 19
4.4. Riesgos 20

5 Conclusiones de la etapa 21

Glosario 23

Referencias 25

A Catálogo de requerimientos 26
A.1. M1 — Recepción multiprotocolo 26
A.2. M2 — Normalización de eventos y telemetría 27
A.3. M3 — Inventario y aprovisionamiento de dispositivos . . . 29
A.4. M4 — Supervisión de salud de enlace 29
A.5. M5 — Persistencia y consulta 30
A.6. M6 — Alertas y notificación 31
A.7. M7 — Seguridad, identidad y multi-tenencia 32
A.8. M8 — Interfaz de administración 33
A.9. Requerimientos no funcionales 33

B Backlog completo 36
B.1. EP-01 — Infraestructura y entorno de desarrollo 36
B.2. EP-02 — Diseño de arquitectura, red y datos 37
B.3. EP-03 — Especificación del contrato de trama y adaptadores 37
B.4. EP-04 — Recepción multiprotocolo 38
B.5. EP-05 — Normalización y máquina de estados 39
B.6. EP-06 — Persistencia y consulta 39
B.7. EP-07 — Inventario y aprovisionamiento 40
B.8. EP-08 — Supervisión de salud de enlace 40
B.9. EP-09 — Seguridad y multi-tenencia 41
B.10. EP-10 — Interfaz de administración 42

B.11. EP-11 — Alertas y notificación 42
B.12. EP-12 — Modelo de despliegue y análisis de costos 43
B.13. EP-13 — Validación y pruebas 43
B.14. EP-14 — Documentación y entregables académicos 44
B.15. Resumen del esfuerzo 44

C Evaluación detallada **45**
C.1. Pros y contras de cada mecanismo de transporte 45
C.2. Identificación de dispositivos 46
C.3. Arquitecturas de nodo consideradas (lado del compañero) 46
C.4. Evaluación de la tecnología de backend 46
C.5. Evaluación de la tecnología de interfaz (frontend) 47
C.6. Presupuesto de recursos frente al referente 48
C.7. Costos de alojamiento (preliminar; el detalle es OE3) 49

D Camino descartado: Contact ID **50**

Índice de figuras

1.1. Árbol de problemas. Las cinco causas —C1 formatos dispares, C2 hardware atado, C3 costo y nube ajena, C4 sin personal técnico, C5 sin visibilidad— provocan el problema central, que produce los cinco efectos. 2

2.1. Los protocolos se apilan, no compiten. Cada nivel responde una pregunta distinta, y la elección en un nivel no condiciona la de los otros. 8
2.2. Patrón publicar/suscribir de MQTT. El desacoplamiento entre emisores y receptores es lo que permite añadir consumidores sin tocar los dispositivos. 9
2.3. Cadena de una red LoRaWAN y ubicación del aporte de este trabajo. 11

3.1. El pipeline de tratamiento de tramas, aporte central del trabajo. Cada filtro es independiente y verificable por separado; los tres destinos —almacenamiento, cuarentena y registro de desconocidos— preservan la evidencia sin contaminar las series. 15
3.2. Despliegue: todos los componentes corren como contenedores en un único servidor. La frontera con el trabajo del compañero es el tópico MQTT; el emulador publica en el mismo tópico, de modo que la validación no depende del hardware. 16

Índice de cuadros

1.1. Matriz de trazabilidad. Las cinco causas y los cinco efectos quedan cubiertos por al menos un objetivo. 6

3.1. Módulos de la solución. El panel es uno de ocho módulos: la sustancia de ingeniería está en M1–M7. 13
3.2. Evaluación del mecanismo de transporte (escala 1–5). Los pros y contras en palabras de cada alternativa están en el anexo C. . 14
3.3. Evaluación del formato de trama. 14
3.4. Decisiones de infraestructura. La matriz completa de la tecnología de backend —cinco alternativas y siete criterios— está en el anexo C; el resto, en el repositorio. 17

4.1. Calendario oficial (pauta del 17 de agosto de 2026). Presupuesto a ∼15 h semanales: ∼240 horas. 19
4.2. Registro de riesgos. 20

B.15. Esfuerzo estimado frente al presupuesto. 44

C.1. Comparación de arquitecturas de nodo. La decisión es del trabajo coordinado del compañero; para esta plataforma todas convergen en el tópico MQTT. 46
C.2. Matriz de tecnología de backend (escala 1–5). 47
C.3. Matriz de tecnología de interfaz (escala 1–5). 48
C.4. Presupuesto de memoria estimado. Da origen al requerimiento RNF-17: la plataforma completa debe operar en un servidor de 2 GB de memoria, lo que se verifica empíricamente en la Etapa 4. 49

Resumen

Las organizaciones que ya poseen sensores no logran convertir sus datos en información utilizable: cada dispositivo emite tramas con formatos distintos, las plataformas comerciales operan solo con el hardware de su proveedor, y no existe visibilidad del estado de la conectividad. Este trabajo desarrolla una plataforma web para el registro, monitoreo y gestión de sensores que recibe tramas de telemetría continua a través de TCP/IP —con MQTT como vía principal y HTTP como vía secundaria—, las normaliza a un modelo interno único mediante un pipeline de seis filtros con cuarentena y adaptadores por formato, almacena las mediciones como series de tiempo, supervisa el estado de cada dispositivo y presenta la información en un panel de control, sobre un caso de validación en el Campus Dr. Luis Rivas del Canto de la Universidad Católica de Temuco. El contrato nativo de entrada es el estándar abierto SenML (RFC 8428). Esta primera etapa define el problema mediante un árbol de causas y efectos, formula un objetivo general y cinco específicos con matriz de trazabilidad, cataloga 68 requerimientos justificados por fuente, evalúa cinco mecanismos de transporte y cuatro formatos de trama mediante matrices de criterios ponderados —documentando además el descarte fundamentado del protocolo Contact ID evaluado en la fase previa— y construye un backlog de 63 elementos cuya verificación numérica delimitó el compromiso al núcleo demostrativo.

Palabras Clave: Internet de las Cosas, monitoreo de sensores, telemetría, MQTT, SenML, normalización de tramas, series de tiempo, monitoreo de sensores, La Araucanía.

Capítulo 1

Introducción

1.1. Problema

La Región de La Araucanía concentra actividad agrícola, agroindustrial y pymes con instalaciones dispersas donde el monitoreo remoto tiene valor operativo directo. Adquirir sensores dejó de ser la barrera: un nodo con radio cuesta entre 5 y 30 dólares y la banda de 915–928 MHz es de uso libre en Chile. La dificultad se desplazó hacia el destino de los datos: el patrón dominante es la **telemetría continua** —lecturas cada 5 a 30 minutos— que debe recibirse, entenderse, almacenarse y vigilarse.

> **Problema central:** las organizaciones que ya poseen sensores no logran convertir los datos que estos generan en información utilizable, porque carecen de una plataforma que unifique la recepción, la normalización y la gestión de dispositivos que emiten tramas con formatos distintos.

El análisis identifica cinco causas: **C1** fragmentación de formatos de trama entre fabricantes; **C2** acoplamiento entre hardware y plataforma; **C3** costo y dependencia de plataformas comerciales en la nube; **C4** ausencia de capacidad técnica interna; y **C5** falta de visibilidad del estado de la conectividad — *el silencio de un sensor es indistinguible de la ausencia de novedades*. La figura 1.1 muestra el árbol completo. Para facilitar su recuerdo, en adelante se citan con un apodo: **C1 formatos**, **C2 hardware**, **C3 nube**, **C4 técnicos**, **C5 visibilidad**.

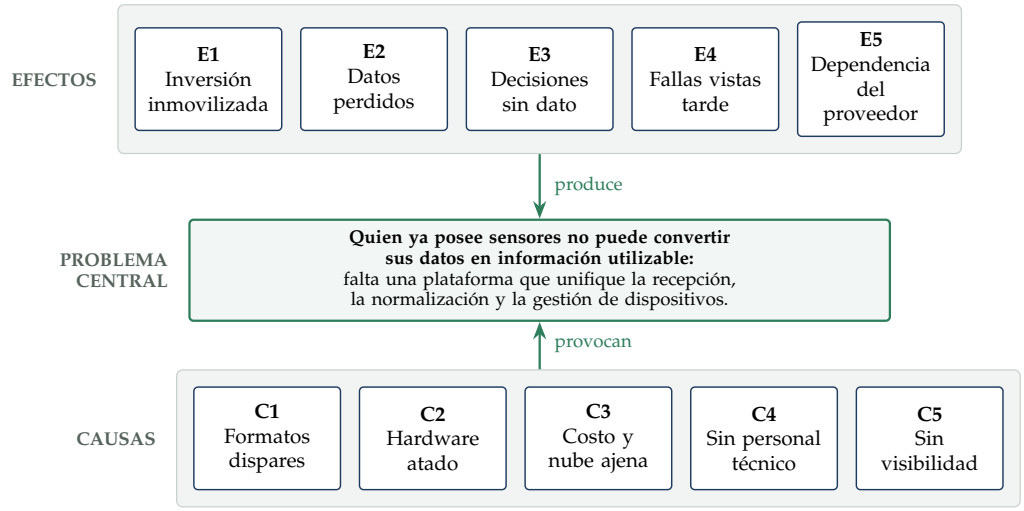


Figura 1.1. Árbol de problemas. Las cinco causas —C1 formatos dispares, C2 hardware atado, C3 costo y nube ajena, C4 sin personal técnico, C5 sin visibilidad— provocan el problema central, que produce los cinco efectos.

La **Universidad Católica de Temuco** es la organización destinataria y parte interesada principal —lo que la rúbrica denomina «propietario del proyecto» en el sentido de dueño del problema, no del trabajo—: dispone de sensores y requiere una plataforma donde registrarlos y monitorearlos. El caso de validación concreto es el **Campus Dr. Luis Rivas del Canto**, sede de la Facultad de Ingeniería, donde en paralelo se despliega una red de sensores que esta plataforma recibirá y gestionará (sección 3.5). El levantamiento directo con las partes interesadas está pendiente y es la acción prioritaria del período; la brecha se declara en lugar de omitirse.

Delimitación. El trabajo abarca la plataforma: recepción de tramas por MQTT y HTTP, normalización a un modelo interno, almacenamiento de series de tiempo, supervisión del estado de los dispositivos, alertas y panel de control, junto con el diseño de red y despliegue asociados. La electrónica de los nodos y la rama de radio LoRa corresponden al trabajo coordinado de un compañero; **la frontera contractual es el tópico MQTT** donde se publican las tramas (sección 3.5), y un emulador de nodo garantiza que la validación de esta plataforma no dependa del

hardware. Quedan fuera la fabricación de sensores, la analítica predictiva y el despliegue productivo a escala.

1.2. Estado del arte

Oferta regional. IoT3 (IoT3, 2026) y ControlTec (ControlTec SpA, 2026) ofrecen monitoreo llave en mano acoplado a su propio hardware: un sensor ya instalado de otra marca no se integra. **Verticales nacionales.** Instacrops (Instacrops, 2026) y WiseConn (WiseConn, 2026) son plataformas agrícolas chilenas maduras, también con hardware y suscripción propios, orientadas a fruticultura de exportación. **SaaS internacionales.** Ubidots y ThingSpeak cobran por dispositivo y alojan los datos fuera del país.

El referente de código abierto es ThingsBoard (ThingsBoard, 2026a): plataforma IoT autoalojable con paneles, reglas y soporte MQTT/HTTP/CoAP. Se evaluó como posible base de este trabajo y se descartó con tres razones: exige recursos considerables (Java, 4 GB o más de memoria recomendada), su configuración —cadenas de reglas, decodificadores de carga útil— requiere personal técnico que las organizaciones destinatarias no poseen (precisamente la causa C4), y su interfaz y documentación están en inglés. A ello se suma el licenciamiento: funciones centrales para este caso —las *Platform Integrations*, incluida la integración con ChirpStack para la rama LoRa, y la personalización de marca— pertenecen a la **edición Professional, de pago** (ThingsBoard, 2026b). La edición Community permite conectar un servidor LoRaWAN de forma gratuita vía MQTT, pero configurando manualmente el puente, las cadenas de reglas y los decodificadores de carga útil: precisamente el trabajo técnico del que carecen las organizaciones destinatarias, con lo que la alternativa gratuita reintroduce la causa C4. Conviene notar que ThingsBoard aplica el mismo patrón de entrada que esta plataforma: exige su propio formato JSON (`{"ts","values"}`) en su tópico `v1/devices/me/telemetry`, y todo formato distinto requiere conversores (ThingsBoard, 2026a). La diferencia de este trabajo es que su contrato nativo es un **estándar IETF** y no un formato definido por el producto. Por último, el trío Node-RED + InfluxDB + Grafana permite prototipos de monitoreo, pero es un conjunto de piezas y no un producto: sin aprovisionamiento de

dispositivos ni modelo de usuarios coherente, cada instalación resulta artesanal.

Conviene además distinguir esta plataforma de un **panel de visualización simple**: mostrar en pantalla que las tramas llegan es una función de validación de una red de sensores, no el objeto de este trabajo. El aporte aquí es la capa que existe *antes* de cualquier gráfico —recepción multiprotocolo, pipeline de normalización, supervisión del estado de cada dispositivo, series de tiempo y alertas—; el panel es solo uno de sus ocho módulos. Un sistema que se limitara a graficar datos ya procesados no resolvería ninguna de las cinco causas del problema.

En la fase previa de este mismo trabajo se evaluó como eje el protocolo de la industria de alarmas Contact ID (SIA DC-05); el profesor guía lo descartó por estar orientado a eventos discretos y no a telemetría continua. El análisis completo y las razones quedan documentados en el anexo D como evidencia del camino evaluado.

1.3. Objetivos

1.3.1. Objetivo general

- Desarrollar una plataforma web para el registro, monitoreo y gestión de datos de sensores, que reciba las tramas de distintos sensores a través de TCP/IP, las normalice a un modelo interno único, almacene las mediciones como series de tiempo, supervise el estado de cada dispositivo y presente la información en un panel de control, validada sobre un caso de uso real en el Campus Dr. Luis Rivas del Canto de la Universidad Católica de Temuco al término del semestre académico 2026.

1.3.2. Objetivos específicos

- **OE1** — Caracterizar la situación de gestión de datos de sensores en la organización destinataria, mediante entrevistas semiestructuradas, revisión de literatura y análisis de la oferta nacional e internacional, para obtener un catálogo de

requerimientos íntegramente trazable a las causas del problema, al 4 de septiembre de 2026.

- **OE2** — Evaluar y seleccionar el mecanismo de transporte y el formato de trama, y especificar el contrato de entrada con sus adaptadores y política de cuarentena, mediante matrices de criterios ponderados, para obtener el documento de especificación de la capa de recepción, al 2 de octubre de 2026.
- **OE3** — Diseñar la arquitectura de la plataforma, su modelo de datos y su despliegue con costo cuantificado, mediante la fase *Design* de PPDIOO y la comparación de al menos tres alternativas de alojamiento, para obtener los diagramas, el modelo y la recomendación justificada, al 2 de octubre de 2026.
- **OE4** — Implementar la plataforma —recepción por MQTT y HTTP, pipeline de normalización, almacenamiento, supervisión y panel—, mediante desarrollo incremental guiado por el backlog y versionado en git, para obtener un prototipo funcional con al menos dos vías de recepción operativas y un sensor real de extremo a extremo, al 4 de diciembre de 2026, con el 50 % verificable al 30 de octubre.
- **OE5** — Validar la plataforma en condiciones que replican el entorno operativo, mediante pruebas funcionales, de carga y de recuperación con comparaciones antes y después, para obtener un informe de validación que acredite el cumplimiento de los requerimientos, al 4 de diciembre de 2026.

Objetivo	Causas	Efectos	Fase PPDIOO	Etapa
OE1 Caracterizar y levantar	C4 (diagnostica C1–C5)	E1, E3	Prepare	1
OE2 Contrato de recepción	**C1**	E2, E5	Plan, Design	2
OE3 Arquitectura y despliegue	C2, C3, C5	E1, E4, E5	Design, Plan	2
OE4 Implementar la plataforma	C1, C2, C4	E1–E3	Implement	3, 4
OE5 Validar en entorno realista	C5	E4	Operate, Optimize	4

Cuadro 1.1. Matriz de trazabilidad. Las cinco causas y los cinco efectos quedan cubiertos por al menos un objetivo.

1.4. Autoría y licenciamiento

Este trabajo de título, su documentación y el código de la plataforma son de **autoría de Vicente Ricardo Ramírez Oñate**. La universidad es la organización destinataria del producto, no la titular de la obra. Es decisión del autor publicar el resultado bajo una licencia abierta al término del trabajo; la licencia concreta se fija en la Etapa 2 junto con el modelo de despliegue.

1.5. Estructura del documento

El capítulo 2 establece los antecedentes técnicos. El capítulo 3 presenta el análisis de requerimientos, las evaluaciones de transporte y formato, y la arquitectura de recepción. El capítulo 4 expone la metodología y la planificación verificada. Los anexos contienen el catálogo completo de requerimientos, el backlog, el detalle de las evaluaciones y la evidencia del camino descartado.

Capítulo 2

Antecedentes

2.1. Los protocolos se apilan, no compiten

Al enumerar «WiFi», «MQTT» y «SenML» parece tratarse de alternativas entre las cuales elegir. Operan en niveles distintos: la capa **física** define cómo viajan los bits (WiFi, LoRa, cable); **TCP/IP** define direcciones y entrega entre máquinas; el **mecanismo de mensajería** define cómo viaja cada mensaje (MQTT, HTTP); y el **formato** define qué dice el mensaje (SenML, JSON). La figura 2.1 resume la escalera. La independencia entre niveles es lo que hace posible la propuesta: **la misma trama llega por WiFi, por cable o por la rama de radio, y la plataforma la interpreta de forma idéntica**.

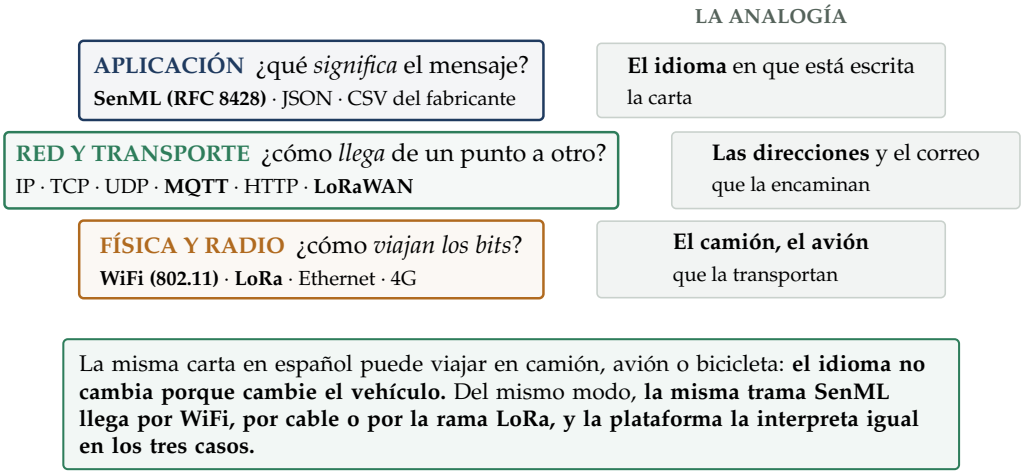


Figura 2.1. Los protocolos se apilan, no compiten. Cada nivel responde una pregunta distinta, y la elección en un nivel no condiciona la de los otros.

Una distinción que evita confusiones frecuentes: **HTTP no es una alternativa a TCP/IP sino un protocolo que corre sobre él**, igual que MQTT. Cuando se critica a HTTP por sus cabeceras —cientos de bytes de metadatos por cada decenas de bytes de dato útil— se critica al protocolo de mensajería, no a la red que hay debajo.

2.2. Mensajería MQTT

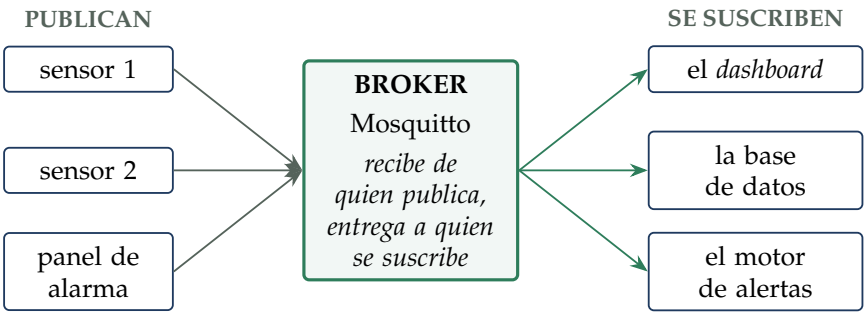
MQTT (*Message Queuing Telemetry Transport*) aplica el patrón publicar/suscribir a través de un intermediario llamado *broker* (figura 2.2): los dispositivos publican en *tópicos* y los consumidores suscritos reciben los mensajes empujados por el broker en el momento, sin consultar. Cuatro propiedades resultan determinantes para este trabajo:

1. **El dispositivo inicia la conexión saliente** y la mantiene: no se abren puertos hacia los sensores y el esquema funciona tras NAT y cortafuegos (requerimiento RNF-10).

2. **Calidad de servicio (QoS)** integrada en tres niveles (0: a lo más una vez; 1: al menos una vez; 2: exactamente una vez), sin lógica de reintento propia.

3. **El testamento (*Last Will*)**: el dispositivo deja registrado al conectarse un mensaje que el broker publica automáticamente si aquel se desconecta sin despedirse — la detección de caída resulta inmediata y sin costo de desarrollo.

4. **Cabeceras de 2 bytes**, frente a los cientos de bytes de HTTP: responde la objeción de sobrecarga para dispositivos que reportan cada pocos minutos.



Los mensajes se organizan en **tópicos**, con forma de ruta:
`sitio/temuco/invernadero-2/sensor-007/temperatura`

Quien se suscribe a `sitio/temuco/#` recibe todo lo del sitio **sin necesidad de saber qué sensores existen**. Añadir un consumidor nuevo no obliga a modificar ningún sensor.

Figura 2.2. Patrón publicar/suscribir de MQTT. El desacoplamiento entre emisores y receptores es lo que permite añadir consumidores sin tocar los dispositivos.

Se emplea **Mosquitto** como broker: implementación en C de consumo mínimo (∼10 MB de memoria), madura, con TLS y autenticación por cliente. Las alternativas de clúster (EMQX, HiveMQ) están dimensionadas para millones de conexiones y resultan desproporcionadas (sección 3.6).

2.3. El formato de trama: SenML

SenML (*Sensor Measurement Lists*, RFC 8428 (Jennings et al., 2018)) es el estándar de la IETF —organismo cuyas normas son públicas y gratuitas— para representar listas de mediciones. Cada trama declara la identidad del dispositivo, el tiempo base y una lista de mediciones con nombre, unidad estándar y valor:

```
[ {"bn":"urn:dev:mac:a4cf12ff001a:", "bt":1756857600},
  {"n":"temperatura", "u":"Cel", "v":23.4},
  {"n":"bateria",     "u":"%EL", "v":87} ]
```

La identidad viaja dentro de la trama (`bn`) y se cruza con el registro de dispositivos de la plataforma. Sobre WiFi y cable el tamaño ($\sim$100–200 bytes) es irrelevante y la legibilidad facilita la depuración; la compactación binaria solo se justifica en la rama de radio, donde el servidor de red la expande antes de publicarla.

2.4. La rama de radio: LoRa, LoRaWAN y ChirpStack

LoRa es la modulación de radio de largo alcance y bajo consumo (capa física); **LoRaWAN** es el protocolo de red sobre ella: registro de nodos, cifrado AES-128 de extremo a extremo y entrega al servidor (LoRa Alliance, 2021). En Chile se usa la banda libre de 915–928 MHz (Certificación Telecom, 2024). El *gateway* capta la radio y la reenvía por IP sin poder descifrarla; **ChirpStack** —el servidor de red, un contenedor más en el servidor de la plataforma— descifra, elimina duplicados, administra los nodos y **publica las mediciones en MQTT**: la rama de radio converge así a la vía principal sin desarrollo adicional (figura 2.3). Esta rama es opcional y corresponde al trabajo coordinado del compañero.

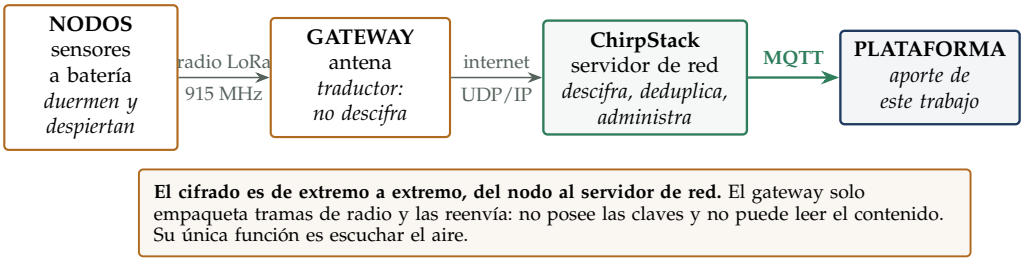


Figura 2.3. Cadena de una red LoRaWAN y ubicación del aporte de este trabajo.

2.5. Persistencia de series de tiempo

Cincuenta sensores reportando cada cinco minutos generan del orden de cinco millones de registros anuales. Se emplea **TimescaleDB**, extensión de PostgreSQL que conserva el modelo relacional y el SQL estándar, añadiendo particionado automático por tiempo, agregación por intervalos y políticas de retención; frente a InfluxDB evita operar dos motores (sección 3.6).

2.6. Seguridad del transporte

TLS (*Transport Layer Security*) es el cifrado del canal —el mismo del candado de HTTPS—: establece un túnel cifrado (AES en su interior) antes de intercambiar datos, y se configura en Mosquitto y en la API sin desarrollo propio. La **identidad** de cada dispositivo se resuelve con dos piezas complementarias: su identificador (por ejemplo la dirección MAC, que dice quién dice ser) y su **credencial** (un secreto emitido al darlo de alta, que lo demuestra). En la rama de radio, LoRaWAN aporta AES-128 de fábrica. Nunca se diseña cifrado propio.

Capítulo 3

Especificación: análisis de requerimientos

3.1. Metodología del levantamiento

Cada requerimiento declara su fuente: **[PO]** solicitud de la organización destinataria —«propietario del proyecto» en la terminología de la rúbrica— vía profesor guía; **[EM]** estudio de mercado; **[BT]** base teórica (RFC 8428, ISO/IEC 25010 (ISO/IEC, 2011), SWEBOK (IEEE Computer Society, 2025)); **[DP]** derivación del problema. **Brecha declarada:** falta la fuente [ENT], entrevistas directas; su cierre es la acción prioritaria del período.

3.2. Síntesis del catálogo

Se catalogan **68 requerimientos**: 52 **funcionales** (RF: lo que el sistema debe *hacer*) en ocho módulos, y 16 **no funcionales** (RNF: cómo debe *comportarse* — rendimiento, seguridad, mantenibilidad) clasificados según ISO/IEC 25010— con prioridad MoSCoW (49 *Must*, 16 *Should*, 3 *Could*). El catálogo completo está en el anexo A.

Módulo	Responsabilidad	Req.	Causas
M1	Recepción multiprotocolo (MQTT, HTTP, adaptadores)	10	C1, C2
M2	Normalización al modelo interno (el pipeline)	7	C1, C5
M3	Registro y aprovisionamiento de dispositivos	6	C4, C5
M4	Supervisión de salud (¿quién dejó de reportar?)	7	**C5**
M5	Persistencia y consulta de series de tiempo	5	C1, C3, C4
M6	Alertas y notificación	4	C5
M7	Seguridad, identidad y multi-tenencia	7	C1, C3, C4
M8	Panel de control (la vista, no el todo)	6	C1, C2, C4, C5

Cuadro 3.1. Módulos de la solución. El panel es uno de ocho módulos: la sustancia de ingeniería está en M1–M7.

Requerimientos que anclan el trabajo al dominio de redes: **RNF-09** (dispositivos en segmento de red separado), **RNF-10** (solo conexiones salientes desde los nodos), **RNF-12** (añadir una vía de recepción no modifica el núcleo) y **RNF-16** (la trama nativa cumple SenML). Y los que sustentan la diferenciación: **RF-04** (adaptadores por formato), **RF-24/27/30** (supervisión de salud) y **RF-34** (exportación en formato abierto).

3.3. Evaluación del mecanismo de transporte

Cinco alternativas sobre seis criterios. Los pesos derivan del problema: la plataforma debe saber qué llegó y qué no (25 %), detectar dispositivos caídos (20 %, causa C5), exigir poco al nodo (20 %, lo implementa el compañero), integrarse con poco esfuerzo al backend (15 %), pesar poco por mensaje (10 %, la objeción a las cabeceras) y funcionar tras NAT sin abrir puertos (10 %, RNF-10).

	Entrega confiable 25 %	Detecta caídos 20 %	Costo en nodo 20 %	Facilidad backend 15 %	Sobrecarga mensaje 10 %	Tras NAT 10 %	Total
MQTT	**5**	**5**	4	**5**	**5**	**5**	**4,85**
HTTP/REST	3	2	4	**5**	2	**5**	3,45
CoAP	3	2	4	2	**5**	4	3,05
WebSocket	2	3	3	3	4	**5**	3,05
Socket TCP propio	1	2	**5**	1	**5**	**5**	2,60

Cuadro 3.2. Evaluación del mecanismo de transporte (escala 1–5). Los pros y contras en palabras de cada alternativa están en el anexo C.

Decisión: MQTT como vía principal —QoS y *Last Will* resuelven de fábrica la entrega confiable y la detección de caída— y HTTP/REST como vía secundaria para integraciones. El navegador nunca toca MQTT: consulta la API REST del backend. El socket TCP propio se descarta explícitamente: lograría la mínima sobrecarga a costa de reinventar sesiones, confirmaciones y formato.

3.4. Evaluación del formato de trama

Alternativa	A favor	En contra
JSON libre	Flexible, universal	**Reproduce la causa C1**: cada fuente con sus campos
SenML (RFC 8428)	Estándar IETF abierto; identidad, unidades y tiempo resueltos; legible	Algo más extenso que un JSON ad hoc
Binario propio	Mínimo peso	Indocumentado, difícil de depurar; solo se justifica en radio
CBOR	Compacto y estándar	Ilegible a ojo; librería adicional en el nodo

Cuadro 3.3. Evaluación del formato de trama.

Decisión: SenML como contrato nativo. Lo que no llegue en SenML entra mediante **adaptadores por formato** (uno por formato de fuente,

no por sensor); la trama malformada de un dispositivo conocido va a **cuarentena** —cruda, con su motivo, sin contaminar las series—; el dispositivo no registrado se **rechaza y se registra el intento**. Este es el mismo patrón de entrada que aplica ThingsBoard con su formato propio (ThingsBoard, 2026a); la diferencia es que aquí el contrato nativo es un estándar abierto.

3.5. Arquitectura de recepción y frontera del trabajo

El aporte central es el **pipeline de tratamiento de tramas** (figura 3.1): seis filtros independientes y verificables por separado. La identificación de dispositivos combina el **registro** (al dar de alta: identidad, tipo, ubicación, credencial) con la **autoidentificación** de cada trama (el campo `bn`); el filtro de validación cruza ambos.

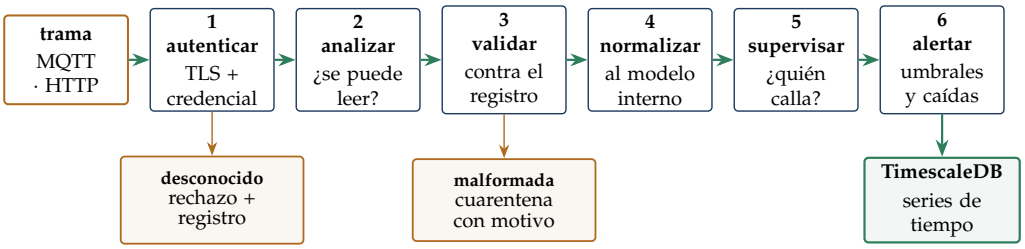


Figura 3.1. El pipeline de tratamiento de tramas, aporte central del trabajo. Cada filtro es independiente y verificable por separado; los tres destinos —almacenamiento, cuarentena y registro de desconocidos— preservan la evidencia sin contaminar las series.

El despliegue completo corre como contenedores en un único servidor (figura 3.2), con reinicio automático ante fallas. **La frontera con el trabajo del compañero es el tópico MQTT**: todo lo anterior (sensor, nodo, radio) es de él; todo lo posterior (broker, pipeline, base de datos, API, panel), de este trabajo. Un **emulador de nodo** publica tramas en el mismo tópico, de modo que la validación de la plataforma no depende del hardware ni de terceros. En la rama LoRa, el servidor de red (ChirpStack) correrá en el servidor de la plataforma —aunque algunos gateways lo

traen embebido— para administrar nodos y claves desde un solo punto; el decodificador de binario a JSON es responsabilidad del trabajo del compañero, y su JSON de tres campos se convierte al contrato SenML mediante el primer adaptador real de la capa de recepción.

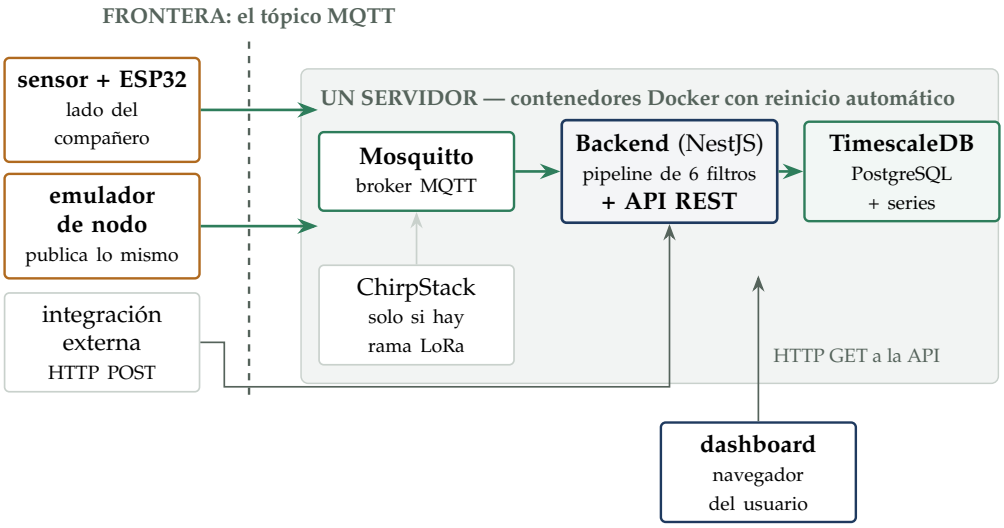


Figura 3.2. Despliegue: todos los componentes corren como contenedores en un único servidor. La frontera con el trabajo del compañero es el tópico MQTT; el emulador publica en el mismo tópico, de modo que la validación no depende del hardware.

3.6. Evaluación de infraestructura

Componente	Selección	Razón determinante frente a alternativas
Broker MQTT	Mosquitto	C, ∼10 MB de RAM, TLS y autenticación; EMQX/HiveMQ apuntan a millones de conexiones: desproporcionados
Persistencia	TimescaleDB	Un solo motor relacional y de series; InfluxDB obligaría a operar dos bases
Backend	TypeScript + NestJS	Tipos de la trama definidos una sola vez para backend y panel; módulos 1:1 con M1–M8; OpenAPI automático
Rama LoRa	ChirpStack	Autoalojable; publica en MQTT: converge a la vía principal
Despliegue	Contenedores Docker	Reproducible con un comando; reinicio automático; la Etapa 4 exige demostrar en condiciones realistas
Interfaz (frontend)	Svelte + uPlot	Artefacto liviano sin *virtual DOM*, coherente con la ligereza objetivo; gráficos de series de tiempo de pocos kilobytes; se confirma en la Etapa 2 (anexo C)

Cuadro 3.4. Decisiones de infraestructura. La matriz completa de la tecnología de backend —cinco alternativas y siete criterios— está en el anexo C; el resto, en el repositorio.

Un requerimiento no funcional propio del enfoque es **RNF-17**: la plataforma completa debe operar en un servidor de 2 GB de memoria —frente a los 4 GB o más que exige el referente—, lo que se estima en el anexo C y se mide en la Etapa 4.

Restricciones: RE-01 dedicación ∼15 h semanales; RE-02 proyecto individual coordinado con el trabajo del compañero; RE-03 solo componentes abiertos; RE-05 umbral de aprobación del 70 %; RE-06 hardware no confirmado, mitigado con el emulador.

Capítulo 4

Metodología y planificación

4.1. Marco metodológico

Se adopta **PPDIOO** (Cisco) como ciclo de vida de ingeniería, por tres razones: es el marco propio del dominio de redes, corresponde a la especialidad del profesor guía, y sus fases coinciden con las cuatro etapas de evaluación (Prepare/Plan $\rightarrow$ Etapa 1; Design $\rightarrow$ Etapa 2; Implement $\rightarrow$ Etapa 3; Operate/Optimize $\rightarrow$ Etapa 4).

Se descarta Scrum como marco de gestión de equipo —el proyecto es unipersonal y sus ceremonias carecen de sentido— pero **se conservan sus artefactos de planificación**: el backlog con épicas, historias de usuario, criterios de aceptación y estimaciones. No es solo preferencia metodológica: la pauta asigna el 20 % de esta etapa al backlog y mide el avance de las siguientes como porcentaje de tareas completadas. PPDIOO define qué fases hay; el backlog, cómo se mide el avance.

Los estándares de referencia siguen al SWEBOK v4 (IEEE Computer Society, 2025) en sus áreas de requerimientos (cap. 1), arquitectura y diseño (caps. 2–3), pruebas (cap. 5) y gestión de configuración (cap. 7): todo el trabajo se versiona en un repositorio git con mensajes descriptivos, y la retroalimentación del profesor guía se registra en bitácora con trazabilidad indicación $\rightarrow$ cambio $\rightarrow$ evidencia.

4.2. Calendario oficial

Etapa	Sprint	Entrega
1 — Problema, objetivos, requerimientos	17 ago – 4 sep	**4 de septiembre**
2 — Diseño e implementación inicial	7 sep – 2 oct	2 de octubre
3 — Mejoras, pruebas, validación preliminar	5 – 30 oct	30 de octubre
4 — Validación final e informe	2 nov – 4 dic	4 de diciembre

Cuadro 4.1. Calendario oficial (pauta del 17 de agosto de 2026). Presupuesto a ∼15 h semanales: ∼240 horas.

4.3. Backlog y verificación de viabilidad

El backlog contiene **63 elementos** —14 épicas, 25 historias de usuario y 38 tareas técnicas— estimados en horas ideales, ajustados tras el pivote del 2 de septiembre: los elementos ligados al protocolo descartado se reformularon hacia el contrato SenML y el pipeline, con un efecto neto de −10 horas en el núcleo demostrativo (∼155 h). El backlog completo está en el anexo B.

> **Criterio de demostración:** el prototipo demuestra la tesis si recibe tramas por al menos dos vías (MQTT y HTTP), las normaliza al modelo interno, detecta que un dispositivo dejó de reportar, lo presenta en el panel con el estado de cada dispositivo, y permite exportar los datos en formato abierto.

La verificación numérica —sumando programáticamente las estimaciones— arrojó el hallazgo de mayor consecuencia práctica de la etapa: **el backlog completo (∼265 h) excede el presupuesto y no es ejecutable íntegro; el núcleo demostrativo (∼155 h) cabe con holgura** (∼+35 % con el calendario oficial). En consecuencia, **el compromiso del proyecto es el núcleo** y los 30 elementos restantes son alcance condicional, ejecutable solo contra holgura verificada al cierre de cada

etapa. El fundamento: la pauta mide el avance como porcentaje de tareas completadas, de modo que comprometer el backlog íntegro y ejecutar solo el núcleo arrojaría cerca de un 60 % —bajo el umbral—, mientras que comprometer el núcleo y ejecutarlo arroja el 100 %.

La distribución temporal también se corrigió: la asignación inicial concentraba la implementación en la Etapa 3 (162 % de su capacidad) cuando la pauta solo exige allí el 50 %; se trasladaron ∼37 horas del núcleo a la Etapa 4, verificando que el umbral del 50 % se cumple (53 %).

4.4. Riesgos

	Riesgo	**Estado**	**Mitigación**
R1	Hardware de sensores no disponible	Activo	Emulador de nodo por software; el sensor físico demuestra el caso, el emulador la carga
R2	Sin acceso a partes interesadas	Activo	Escalar vía profesor guía; documentar el intento; fuentes secundarias
R3	Ambigüedad de fechas	**Cerrado**	Calendario oficial publicado el 24/08
R4	Alcance excesivo	**Confirmado y controlado**	Núcleo comprometido vs. alcance condicional; revisión de holgura por etapa
R5	Protocolo inadecuado al patrón de uso	**Cerrado**	Se materializó y resolvió: Contact ID descartado el 02/09; el transporte es MQTT y el formato SenML (sección 3.3)

Cuadro 4.2. Registro de riesgos.

Capítulo 5

Conclusiones de la etapa

Sobre la redefinición del enfoque. La etapa incluyó un cambio de eje decidido con el profesor guía: el protocolo Contact ID, evaluado en la fase inicial, resultó adecuado para eventos discretos pero no para el patrón real del problema —telemetría continua—, y fue descartado. El costo del cambio fue acotado precisamente por el método: al estar cada decisión documentada con criterios explícitos, bastó rehacer la evaluación con pesos nuevos (capítulo 3) y el resto del trabajo —metodología, requerimientos, backlog, análisis de competencia— se conservó en su mayor parte. El camino descartado queda como evidencia en el anexo D.

Sobre la selección resultante. MQTT con tramas SenML resuelve con mecanismos de fábrica lo que el problema exige: calidad de servicio para la entrega, testamento (*Last Will*) para la detección inmediata de caídas, conexiones salientes que atraviesan NAT, y un formato de medición estandarizado por la IETF. El aporte de ingeniería del trabajo se concentra en el **pipeline de tratamiento de tramas** —autenticar, analizar, validar, normalizar, supervisar, alertar— con cuarentena y adaptadores por formato: el mismo patrón de entrada que aplican las plataformas existentes, con la diferencia de que el contrato nativo es un estándar abierto y no un formato de producto.

Sobre la viabilidad. La verificación numérica del backlog, recalculada tras el pivote, mantiene la conclusión: el compromiso es el núcleo demostrativo ($\sim$155 h frente a $\sim$240 h de presupuesto), y el resto es alcance condicional.

Trabajo pendiente para cerrar la etapa: las entrevistas a las partes interesadas (criterio 1) y el acuerdo con el compañero sobre los sensores concretos y el tópico de frontera. **Trabajo previsto para la Etapa 2** (fase *Design*): especificación del contrato de recepción (tópicos, esquema, cuarentena, adaptadores), diagramas de arquitectura y red, modelo de datos, modelo de despliegue con costos, e inicio adelantado de la implementación. Para la validación de la Etapa 4 se planifica además un **banco de comparación contra ThingsBoard Community Edition**: instalar ambas plataformas en el mismo servidor y medir consumo de memoria, tiempo de puesta en marcha y número de pasos para dar de alta un sensor y verlo graficado — una comparación empírica y reproducible del ajuste de cada una al contexto destinatario.

Glosario

Adaptador Pieza de la capa de recepción que traduce un formato de trama no nativo al modelo interno. Se escribe uno por formato de fuente, no uno por sensor.

API REST Conjunto de puertas HTTP que el backend ofrece a los consumidores (el panel, los integradores): `GET /dispositivos`, `GET /mediciones`. Vive dentro del backend.

Aprovisionamiento (dar de alta) Registrar un dispositivo en la plataforma antes de que pueda publicar: identidad, tipo, ubicación y emisión de su credencial.

Broker Intermediario de MQTT: recibe de quienes publican y reparte a quienes se suscribieron. **Mosquitto** es la implementación empleada.

ChirpStack Servidor de red LoRaWAN (contenedor propio, solo si la rama de radio se concreta): descifra, elimina duplicados, administra nodos y publica las mediciones en MQTT.

Credencial Secreto emitido al dar de alta un dispositivo, que demuestra su identidad. La dirección MAC identifica; la credencial autentica.

Cuarentena Destino de las tramas malformadas de dispositivos conocidos: se guardan crudas, con su motivo, sin contaminar las series.

Last Will (testamento) Mensaje que el dispositivo deja registrado al conectarse y que el broker publica automáticamente si aquel se desconecta sin despedirse: detección de caída inmediata.

Middleware Software intermedio entre dos sistemas que no se hablan directo (ChirpStack entre la radio y MQTT; la capa de normalización entre los sensores y la base de datos).

MQTT *Message Queuing Telemetry Transport*: protocolo de mensajería publicar/suscribir sobre TCP/IP, con cabecera de 2 bytes, QoS y testamento. La vía principal de recepción.

NAT y puertos El router comparte una dirección pública y bloquea conexiones entrantes no solicitadas. Como el sensor siempre inicia la conexión saliente, no se abre ningún puerto (RNF-10).

Payload Los bytes de dato útil dentro de un mensaje, descontando las envolturas. Aquí: la trama SenML.

Pipeline Línea de montaje de software: pasos encadenados donde la salida de uno es la entrada del siguiente. El de esta plataforma tiene seis filtros (figura 3.1).

QoS *Quality of Service* de MQTT: 0 «a lo más una vez», 1 «al menos una vez», 2 «exactamente una vez».

SenML *Sensor Measurement Lists* (RFC 8428, IETF, gratuito): formato estándar para listas de mediciones con identidad, nombre, unidad, valor y tiempo. El contrato nativo de entrada.

Socket Conexión TCP cruda entre dos programas, sin protocolo de mensajería encima. Evaluado y descartado: obliga a reinventar lo que MQTT ya trae.

TimescaleDB Extensión de PostgreSQL para series de tiempo; sigue siendo SQL relacional.

TLS *Transport Layer Security*: el cifrado del candado de HTTPS (AES en su interior). Se configura en el broker y la API; no se desarrolla.

Tópico La «ruta» de un mensaje MQTT: `sitio/7/dispositivo/12/medicion`. La frontera contractual con el trabajo del compañero.

Referencias

Certificación Telecom. (2024). *LoRa Chile: LoRaWAN 915 MHz AU915, Resolución 737 de Subtel*. Consultado el 21 de agosto de 2026, desde https://certificacion-telecom.cl/tecnologia/lora/

ControlTec SpA. (2026). *Monitoreo y telemetría industrial*. Consultado el 17 de agosto de 2026, desde https://www.controltec.cl/

IEEE Computer Society. (2025). *Guide to the Software Engineering Body of Knowledge (SWEBOK), v4.0*. https://www.computer.org/education/bodies-of-knowledge/software-engineering

Instacrops. (2026). *Plataforma de agricultura de precisión*. Consultado el 2 de septiembre de 2026, desde https://www.instacrops.com/

IoT3. (2026). *Soluciones inteligentes de monitoreo y automatización*. Consultado el 17 de agosto de 2026, desde https://iot3.cl/

ISO/IEC. (2011). *ISO/IEC 25010: Systems and software Quality Requirements and Evaluation (SQuaRE)*. International Organization for Standardization.

Jennings, C., Shelby, Z., Arkko, J., Keränen, A., & Bormann, C. (2018). *Sensor Measurement Lists (SenML)* (RFC 8428). IETF.

LoRa Alliance. (2021). *RP002-1.0.3 LoRaWAN Regional Parameters*. LoRa Alliance. https://lora-alliance.org/wp-content/uploads/2021/05/RP002-1.0.3-FINAL-1.pdf

ThingsBoard. (2026a). *Getting Connected — MQTT Device API Reference*. Consultado el 2 de septiembre de 2026, desde https://thingsboard.io/docs/reference/mqtt-api/

ThingsBoard. (2026b). *Platform Integrations — ThingsBoard Professional Edition*. Consultado el 3 de septiembre de 2026, desde https://thingsboard.io/docs/user-guide/integrations/

WiseConn. (2026). *DropControl: sistema de riego inteligente*. Consultado el 2 de septiembre de 2026, desde https://wiseconn.com/cl/

Apéndice A

Catálogo completo de requerimientos

Cada requerimiento declara su fuente, la causa del problema que aborda y su prioridad MoSCoW. Catálogo actualizado al pivote del 2 de septiembre (contrato SenML sobre MQTT).

A.1. M1 — Recepción multiprotocolo

ID	Requerimiento	Fuente	Causa	Pr.
RF-01	El sistema debe exponer un punto de recepción **HTTP** que acepte tramas de medición en **SenML (RFC 8428)**	[PO] [BT]	C1	**M**
RF-02	El sistema debe poder recibir mediciones de la **rama LoRaWAN** a través del servidor de red, que las publica en MQTT (rama opcional)	[PO] [BT]	C1, C2	**M**
RF-03	El sistema debe recibir eventos publicados en un **broker MQTT**, suscribiéndose a los tópicos configurados por sitio	[PO]	C1, C2	**M**

ID	Requerimiento	Fuente	Causa	Pr.
RF-04	El sistema debe admitir **adaptadores de formato por fuente** que traduzcan al modelo interno tramas que no lleguen en SenML, sin modificar el núcleo	[EM] [BT]	C2	**S**
RF-05	El sistema debe **validar cada trama** contra el esquema SenML y contra el registro del dispositivo, rechazando las inválidas sin contaminar las series	[BT]	C1	**M**
RF-06	El sistema debe **confirmar la recepción** al emisor: nivel de QoS acordado en MQTT y código de respuesta en HTTP	[BT]	C1	**M**
RF-07	El sistema debe **poner en cuarentena** las tramas rechazadas —crudas, con su motivo— y registrar los intentos de dispositivos desconocidos	[DP]	C5	**S**
RF-08	El sistema debe aceptar **múltiples mediciones por trama** (lista SenML con nombre, valor, unidad y tiempo por medición)	[BT]	C1	**M**
RF-09	El sistema debe **preservar los metadatos del transporte** de cada recepción (RSSI, SNR, gateway, identificador del nodo) asociándolos al evento	[BT] [DP]	C5	**M**
RF-10	El sistema debe **descartar duplicados** cuando el mismo evento llegue por más de un camino o gateway	[BT]	C1	**S**

A.2. M2 — Normalización de eventos y telemetría

ID	Requerimiento	Fuente	Causa	Pr.
RF-11	El sistema debe **normalizar toda trama aceptada al modelo interno único** (dispositivo, magnitud, unidad, valor, marca temporal), venga de la vía y formato que venga	[PO] [BT]	C1	**M**
RF-12	El sistema debe mantener **condiciones abiertas y cerradas** por dispositivo: el cruce de umbral abre la condición y la vuelta al rango la cierra	[BT]	C1, C5	**M**
RF-13	El sistema debe **generar eventos por umbral** cuando una medición cruce un límite configurado	[PO]	C1	**M**
RF-14	El sistema debe permitir **configurar umbrales de disparo y de restauración por dispositivo**, con banda de histéresis para evitar oscilación	[BT]	C5	**S**
RF-15	El sistema debe **correlacionar eventos con telemetría** mediante la tripleta de identidad `(ACCT, GG, CCC)` y la marca temporal, para reconstruir el contexto de un evento	[DP]	C1	**S**
RF-16	El sistema debe **normalizar las marcas temporales** a un huso único, tolerando que el nodo no disponga de reloj confiable y usando la hora de recepción cuando corresponda	[BT]	C1	**M**
RF-17	El sistema debe **resolver la identidad del dispositivo** desde los metadatos del transporte cuando el mensaje no la incluya explícitamente	[DP]	C1	**M**

A.3. M3 — Inventario y aprovisionamiento de dispositivos

ID	Requerimiento	Fuente	Causa	Pr.
RF-18	El sistema debe mantener un **inventario de dispositivos** con tipo, ubicación, sitio, zona lógica, transporte de llegada y estado	[PO]	C4, C5	**M**
RF-19	El sistema debe permitir **registrar un dispositivo nuevo** asignándole su tripleta `(ACCT, GG, CCC)` sin intervenir la consola del servidor de red	[PO] [EM]	C4	**M**
RF-20	El sistema debe **rechazar el registro** de un dispositivo cuya tripleta ya exista, y advertir cuando el espacio de direcciones de un sitio se agote	[BT]	C1	**S**
RF-21	El sistema debe permitir **agrupar dispositivos por sitio y por zona lógica**, y presentar la vista agrupada	[PO]	C4	**M**
RF-22	El sistema debe permitir **marcar un dispositivo en mantenimiento**, suprimiendo sus alertas y dejando registro del período	[BT]	C5	**C**
RF-23	El sistema debe registrar **quién realizó cada cambio de configuración de un dispositivo y cuándo**	[BT]	C5	**S**

A.4. M4 — Supervisión de salud de enlace

ID	Requerimiento	Fuente	Causa	Pr.
RF-24	El sistema debe **detectar la pérdida de comunicación** de un dispositivo cuando no reporte durante un número configurable de intervalos, y abrir la condición correspondiente	[PO] [BT]	**C5**	**M**
RF-25	El sistema debe **detectar la restauración** del dispositivo y cerrar la condición, registrando la duración de la interrupción	[BT]	C5	**M**
RF-26	El sistema debe registrar la **última vez que se recibió** comunicación de cada dispositivo	[PO]	C5	**M**
RF-27	El sistema debe **presentar la calidad del enlace** por dispositivo (RSSI, SNR y porcentaje de recepción) cuando el transporte la provea	[BT]	C5	**M**
RF-28	El sistema debe **procesar eventos de batería baja** (código 384) y exponer los dispositivos que requieren mantenimiento	[PO] [BT]	C5	**M**
RF-29	El sistema debe **aprovechar el *keep-alive* y el *Last Will* de MQTT** como evidencia de conexión y aviso inmediato de desconexión	[BT]	C5	**M**
RF-30	El sistema debe presentar un **indicador agregado de salud por sitio**: total de dispositivos, en línea, con condición abierta y sin supervisión	[PO]	C4, C5	**M**

A.5. M5 — Persistencia y consulta

ID	Requerimiento	Fuente	Causa	Pr.
RF-31	El sistema debe **almacenar de forma persistente** todo evento aceptado, conservando el mensaje original íntegro junto a su interpretación	[BT] [DP]	C1	**M**
RF-32	El sistema debe **almacenar series de tiempo** de telemetría con valor, unidad y marca temporal	[PO]	C1	**M**
RF-33	El sistema debe permitir **consultar el historial** de eventos y mediciones de un dispositivo en un rango de fechas	[PO]	C4	**M**
RF-34	El sistema debe permitir **exportar** los datos de un sitio en formato abierto (CSV y JSON)	[PO] [EM]	**C3, C5**	**M**
RF-35	El sistema debe exponer una **API de consulta** que permita a sistemas externos acceder a los datos del sitio	[PO]	C3	**S**

A.6. M6 — Alertas y notificación

ID	Requerimiento	Fuente	Causa	Pr.
RF-36	El sistema debe **notificar** cuando se abra una condición de severidad configurada como notificable	[PO]	C5	**M**
RF-37	El sistema debe permitir **configurar los destinatarios** de notificación por sitio	[PO]	C4	**S**
RF-38	El sistema debe **agrupar notificaciones repetidas** del mismo dispositivo en una ventana temporal, para evitar saturación	[BT]	C5	**S**

ID	Requerimiento	Fuente	Causa	Pr.
RF-39	El sistema debe permitir **reconocer una alerta**, registrando quién y cuándo	[BT]	C5	**C**

A.7. M7 — Seguridad, identidad y multi-tenencia

ID	Requerimiento	Fuente	Causa	Pr.
RF-40	El sistema debe **autenticar toda recepción** mediante credencial por sitio, rechazando mensajes sin credencial válida	[BT]	C1	**M**
RF-41	El sistema debe **aislar los datos entre sitios**: ningún usuario de un sitio puede acceder a datos de otro	[BT] [DP]	C3	**M**
RF-42	El sistema debe **cifrar el transporte** de toda comunicación externa: TLS para HTTP y MQTT (la rama LoRaWAN trae AES-128 de fábrica)	[BT]	C1	**M**
RF-43	El sistema debe **autenticar a los usuarios** de la interfaz de administración con credenciales individuales	[BT]	C4	**M**
RF-44	El sistema debe distinguir al menos **dos roles**: administrador del sitio y usuario de consulta	[PO]	C4	**S**
RF-45	El sistema debe permitir **rotar las credenciales de recepción** de un sitio sin interrumpir el servicio	[BT]	C1	**C**
RF-46	El sistema debe **registrar en bitácora** los accesos y las operaciones sensibles	[BT]	C5	**S**

A.8. M8 — Interfaz de administración

ID	Requerimiento	Fuente	Causa	Pr.
RF-47	La interfaz debe presentar el **inventario de dispositivos** con su estado y transporte de llegada, filtrable por sitio, zona y estado	[PO]	C4	**M**
RF-48	La interfaz debe presentar el **panel de salud del sitio** con semáforo por dispositivo	[PO]	C5	**M**
RF-49	La interfaz debe presentar el **detalle de un dispositivo**: identidad, condiciones abiertas, historial de eventos y serie de mediciones	[PO]	C4	**M**
RF-50	La interfaz debe ofrecer el **asistente de registro** de un dispositivo nuevo	[PO]	C4	**M**
RF-51	La interfaz debe presentar la **vista agrupada por transporte de red**, mostrando cuántos dispositivos llegan por cada tecnología	[PO] [DP]	C1, C2	**S**
RF-52	La interfaz debe ser **utilizable desde un navegador móvil**	[EM]	C4	**S**

A.9. Requerimientos no funcionales

ID	Requerimiento	Verificación	Fuente	Pr.
RNF-01	El sistema debe sostener la recepción de $\geq$ **50 eventos por segundo** sin pérdida	Prueba de carga con generador sintético	[DP]	**M**
RNF-02	El tiempo desde la recepción de un evento hasta su disponibilidad para consulta debe ser $\leq$ **2 segundos** en el percentil 95	Medición instrumentada	[DP]	**M**

ID	Requerimiento	Verificación	Fuente	Pr.
RNF-03	El sistema debe **detectar la pérdida de supervisión** de un dispositivo en $\leq$ **2 intervalos** de reporte configurados	Prueba funcional con nodo silenciado	[PO]	**M**
RNF-04	Una consulta de historial de 30 días de un dispositivo debe responder en $\leq$ **3 segundos**	Medición con volumen sintético	[DP]	**S**
RNF-05	Ningún evento aceptado y confirmado puede perderse ante un reinicio del sistema	Prueba de recuperación ante falla	[BT]	**M**
RNF-06	El sistema debe **reanudar la recepción automáticamente** tras recuperarse de una interrupción, sin intervención manual	Prueba de reinicio	[BT]	**M**
RNF-07	La indisponibilidad de un adaptador de transporte **no debe afectar** la recepción por los demás	Prueba de aislamiento de fallas	[DP]	**S**
RNF-08	Ninguna credencial debe almacenarse en texto claro, ni en base de datos ni en el repositorio de código	Inspección y revisión del repositorio	[BT]	**M**
RNF-09	Los dispositivos IoT deben poder residir en un **segmento de red separado** del segmento de usuarios, con acceso restringido solo a los puertos de recepción	Diseño de red verificado y documentado	[BT]	**M**
RNF-10	El sistema **no debe requerir puertos entrantes abiertos hacia los dispositivos**: la conexión siempre se inicia desde el dispositivo hacia la plataforma	Revisión de arquitectura	[BT]	**M**

ID	Requerimiento	Verificación	Fuente	Pr.
RNF-11	El código debe residir en un **repositorio git** con mensajes de commit descriptivos en la totalidad de ellos	Inspección del historial	[BT]	**M**
RNF-12	Añadir un **transporte nuevo** debe requerir implementar solo un adaptador, sin modificar el núcleo de normalización	Revisión de diseño y prueba de extensión	[DP]	**M**
RNF-13	La especificación del protocolo debe estar **documentada de forma independiente** de la implementación	Existencia del documento	[PO]	**M**
RNF-14	El sistema debe poder **desplegarse completo en un único servidor** con recursos modestos, sin dependencia de servicios propietarios de nube	Despliegue verificado en máquina local	[PO]	**M**
RNF-15	El sistema debe construirse **exclusivamente con componentes de código abierto**, sin licencias de pago	Inventario de dependencias	[PO]	**M**
RNF-16	La trama nativa debe cumplir **SenML (RFC 8428)**; la rama LoRa, de concretarse, debe caber en el payload disponible del plan regional	Validación de esquema + verificación de tamaño	[BT]	**M**

Apéndice B

Backlog completo

Los elementos marcados **(N)** pertenecen al núcleo demostrativo. Actualizado al pivote del 2 de septiembre.

B.1. EP-01 — Infraestructura y entorno de desarrollo

ID	Elemento	Pr.	Etapa	Horas
TT-01	**(N)** Componer el entorno con contenedores: TimescaleDB, Mosquitto y ChirpStack orquestados, con un único comando de arranque	M	2	6
TT-02	**(N)** Inicializar el proyecto TypeScript + NestJS con estructura de módulos correspondiente a M1–M8	M	2	4
TT-03	Configurar verificación automática: tipos, formato y ejecución de pruebas	S	2	3
TT-04	Documentar el procedimiento de puesta en marcha en el repositorio	M	2	1

B.2. EP-02 — Diseño de arquitectura, red y datos

ID	Elemento	Pr.	Etapa	Horas
TT-05	**(N)** Diagrama de arquitectura de componentes, con los ocho módulos y sus dependencias	M	2	5
TT-06	**(N)** Diagrama de topología de red con segmentación de dispositivos IoT (RNF-09) y sentido de las conexiones (RNF-10)	M	2	6
TT-07	**(N)** Modelo de datos entidad-relación con al menos ocho entidades	M	2	6
TT-08	Diagrama de flujo de recepción, desde la recepción hasta la persistencia, para los tres transportes	M	2	4
TT-09	Diagrama de secuencia del aprovisionamiento de un dispositivo nuevo	M	2	3
TT-10	Registro de decisiones de diseño con su justificación	M	2	2

B.3. EP-03 — Especificación del contrato de trama y adaptadores

ID	Elemento	Pr.	Etapa	Horas
TT-11	**(N)** Esquema del modelo interno y contrato del tópico MQTT (nombres, unidades, identidad)	M	2	3
TT-12	**(N)** Catálogo de tipos de sensor y unidades aceptadas, con umbrales de disparo y restauración	M	2	6

ID	Elemento	Pr.	Etapa	Horas
TT-13	**(N)** Especificación del contrato de la API en OpenAPI, generada desde el código	M	2	3
TT-14	Política de QoS, reintento y cuarentena	M	2	2
TT-15	Procedimiento de asignación de identidad y gestión de claves AES por sitio	S	2	2

B.4. EP-04 — Recepción multiprotocolo

ID	Elemento	Pr.	Etapa	Horas
HU-01	**(N) Como integrador**, quiero publicar una trama SenML por MQTT o HTTP, para reportar desde un sensor sin infraestructura intermedia	M	2	6
HU-02	**(N) Como integrador**, quiero que mis eventos LoRaWAN lleguen a la plataforma, para aprovechar los nodos de radio ya instalados	M	3	6
HU-03	**(N) Como integrador**, quiero publicar eventos en MQTT, para integrar dispositivos que ya usan ese transporte	M	3	4
TT-16	**(N)** Validador de esquema SenML y analizador de tramas	M	2	3
HU-04	**Como integrador**, quiero un adaptador para un formato no SenML, para demostrar la extensibilidad de la capa de recepción	S	4	7

B.5. EP-05 — Normalización y máquina de estados

ID	Elemento	Pr.	Etapa	Horas
TT-17	**(N)** Pipeline de normalización al modelo interno (los 6 filtros) con cuarentena	M	3	6
HU-05	**(N) Como administrador de sitio**, quiero que la plataforma sepa qué condiciones están abiertas en cada dispositivo, para no tener que deducirlo del historial	M	3	7
HU-06	**Como administrador de sitio**, quiero configurar umbrales por dispositivo, para que se genere un evento cuando una medición los cruce	M	3	4
TT-18	**(N)** Normalización de marcas temporales, tolerando nodos sin reloj confiable	M	3	2

B.6. EP-06 — Persistencia y consulta

ID	Elemento	Pr.	Etapa	Horas
TT-19	**(N)** Esquema de base de datos y migraciones versionadas	M	2	5
HU-07	**(N) Como usuario de consulta**, quiero ver el historial de eventos y mediciones de un dispositivo en un rango de fechas	M	3	4
HU-08	**(N) Como administrador de sitio**, quiero exportar los datos de mi sitio en CSV y JSON, para no depender de la plataforma para conservarlos	M	3	4

ID	Elemento	Pr.	Etapa	Horas
HU-09	**Como integrador**, quiero consultar los datos del sitio por API, para alimentar sistemas propios	S	4	4

B.7. EP-07 — Inventario y aprovisionamiento

ID	Elemento	Pr.	Etapa	Horas
HU-10	**(N) Como administrador de sitio**, quiero ver el inventario de mis dispositivos con su estado y el transporte por el que llegan	M	3	5
HU-11	**(N) Como administrador de sitio**, quiero registrar un dispositivo nuevo desde la plataforma, para no tener que operar la consola del servidor de red	M	3	5
HU-12	**Como administrador de sitio**, quiero agrupar dispositivos por sitio y zona lógica, para reconocer mi instalación física	M	3	3
HU-13	**Como administrador de sitio**, quiero marcar un dispositivo en mantenimiento, para que no genere alertas mientras lo intervengo	C	4	2

B.8. EP-08 — Supervisión de salud de enlace

ID	Elemento	Pr.	Etapa	Horas
HU-14	**(N) Como administrador de sitio**, quiero que la plataforma detecte que un dispositivo dejó de reportar, porque el silencio de un sensor es indistinguible de que no haya novedades	M	3	7
HU-15	**(N) Como administrador de sitio**, quiero ver la calidad del enlace de cada dispositivo, para anticipar fallas antes de que ocurran	M	3	4
HU-16	**(N) Como administrador de sitio**, quiero un indicador agregado de salud de mi sitio, para saber de un vistazo si algo requiere atención	M	3	4
HU-17	**Como administrador de sitio**, quiero saber qué dispositivos tienen la batería baja, para planificar el mantenimiento	M	3	3

B.9. EP-09 — Seguridad y multi-tenencia

ID	Elemento	Pr.	Etapa	Horas
TT-20	**(N)** Autenticación de la recepción mediante credencial por sitio	M	3	4
TT-21	**(N)** Aislamiento de datos entre sitios, verificado con pruebas	M	3	5
TT-22	**(N)** Cifrado del transporte: TLS para HTTP y MQTT	M	3	4
HU-18	**Como administrador de sitio**, quiero iniciar sesión con mis credenciales y que existan roles distintos de administración y consulta	M	3	3

B.10. EP-10 — Interfaz de administración

ID	Elemento	Pr.	Etapa	Horas
HU-19	**(N) Como administrador de sitio**, quiero un panel de salud del sitio con semáforo por dispositivo	M	3	9
HU-20	**(N) Como administrador de sitio**, quiero la vista de inventario filtrable por sitio, zona, estado y transporte	M	3	7
HU-21	**(N) Como usuario de consulta**, quiero el detalle de un dispositivo con sus condiciones abiertas, historial y serie de mediciones	M	4	7
HU-22	**Como administrador de sitio**, quiero un asistente guiado para registrar un dispositivo nuevo	M	4	4
HU-23	**Como administrador de sitio**, quiero ver mi flota agrupada por tecnología de red, para entender por dónde llega cada cosa	S	4	3

B.11. EP-11 — Alertas y notificación

ID	Elemento	Pr.	Etapa	Horas
HU-24	**Como administrador de sitio**, quiero recibir una notificación cuando se abra una condición crítica	M	4	5
HU-25	**Como administrador de sitio**, quiero configurar los destinatarios de notificación de mi sitio	S	4	3
TT-23	Agrupación de notificaciones repetidas en una ventana temporal	S	4	2

B.12. EP-12 — Modelo de despliegue y análisis de costos

ID	Elemento	Pr.	Etapa	Horas
TT-24	Comparar al menos tres alternativas de alojamiento con criterios ponderados	M	2	4
TT-25	Cuantificar el costo mensual y anual de operación en CLP y USD	M	2	3
TT-26	Redactar la recomendación de despliegue justificada	M	2	1

B.13. EP-13 — Validación y pruebas

ID	Elemento	Pr.	Etapa	Horas
TT-27	**(N)** Pruebas automatizadas del contrato de trama (esquema, cuarentena, adaptadores)	M	3	5
TT-28	**(N)** Emulador de nodo por software, que genera eventos y telemetría sintéticos	M	3	5
TT-29	**(N)** Prueba de carga: sostener ≥ 50 eventos/segundo sin pérdida (RNF-01)	M	4	4
TT-30	Prueba de recuperación: ningún evento confirmado se pierde ante reinicio (RNF-05, RNF-06)	M	4	4
TT-31	Batería de al menos 15 casos de prueba funcionales documentados	M	4	5
TT-32	Validación de extremo a extremo con al menos un sensor físico	M	4	3

B.14. EP-14 — Documentación y entregables académicos

ID	Elemento	Pr.	Etapa	Horas
TT-33	**(N)** Entrevistas a partes interesadas y consolidación de sus declaraciones	M	1	4
TT-34	Armar el documento LaTeX de la Etapa 1 y compilar el PDF de entrega	M	1	5
TT-35	Documento de diseño de la Etapa 2, con todos los diagramas	M	2	6
TT-36	Informe de pruebas y validación preliminar de la Etapa 3	M	3	5
TT-37	Informe final: problema, requerimientos, diseño, implementación, pruebas y conclusiones	M	4	12
TT-38	Bitácora de reuniones y trazabilidad de retroalimentación incorporada	M	1–4	2

B.15. Resumen del esfuerzo

Conjunto	Elementos	Horas
Núcleo demostrativo	33	∼**155**
Alcance condicional	32	∼117
Total del backlog	**65**	∼**272**
Presupuesto disponible (calendario oficial)	—	∼**240**

Cuadro B.15. Esfuerzo estimado frente al presupuesto.

Apéndice C

Evaluación detallada de transporte y formato

C.1. Pros y contras de cada mecanismo de transporte

MQTT (4,85). A favor: publicar/suscribir desacopla emisores de consumidores; QoS en tres niveles sin lógica propia; el testamento (*Last Will*) hace inmediata la detección de caída; cabecera de 2 bytes; conexión saliente persistente que atraviesa NAT; nació para IoT en redes inestables. En contra: exige operar un broker (Mosquitto: un contenedor, consumo mínimo); el navegador no lo habla nativo (irrelevante: el panel consulta la API).

HTTP/REST (3,45). A favor: universal; sin broker; trivial de probar. En contra: cientos de bytes de cabeceras por decenas de bytes de dato; sin noción de «cliente conectado» (la caída solo se infiere por silencio); una conexión por envío. Rol: vía secundaria de integración.

CoAP (3,05). El «HTTP compacto» sobre UDP para nodos mínimos. Digno técnicamente; ecosistema pequeño, UDP a veces filtrado, sin ventaja sobre MQTT en este caso.

WebSocket (3,05). Canal persistente pensado para navegadores; para sensores obliga a construir encima temas, confirmaciones y testamento: lo que MQTT trae de fábrica. Útil como túnel de MQTT para paneles en vivo.

Socket TCP propio (2,60). Máxima ligereza y control; a cambio, reinventar sesiones, reconexión, confirmaciones y formato. Se descarta con argumento: meses de trabajo para un resultado inferior a MQTT.

C.2. Identificación de dispositivos

El registro (aprovisionamiento) declara identidad, tipo, ubicación y credencial; cada trama se autoidentifica con el campo `bn`. La validación cruza ambos: dispositivo desconocido se rechaza y se registra el intento; trama malformada de dispositivo conocido va a cuarentena; unidad que no corresponde al tipo declarado va a cuarentena. Un sensor nuevo no exige tocar código: se registra y sus tramas ya se entienden.

C.3. Arquitecturas de nodo consideradas (lado del compañero)

	Puente ESP32	Mini-PC de campo	Sensor con red propia
Costo por punto	∼5–10 USD	∼80–200 USD	según fabricante
Consumo	bajo, admite batería	alto, enchufado	variable
Flexibilidad	cualquier sensor serial	máxima (Python, SSH)	la que dio el fabricante
Rol propuesto	nodo estándar	banco de pruebas	caso de integración HTTP

Cuadro C.1. Comparación de arquitecturas de nodo. La decisión es del trabajo coordinado del compañero; para esta plataforma todas convergen en el tópico MQTT.

C.4. Evaluación de la tecnología de backend

Cinco alternativas sobre siete criterios. El peso mayor es la experiencia previa del desarrollador (25 %), porque en un proyecto individual con plazos cerrados el tiempo de aprendizaje se descuenta del de construcción; le siguen la correspondencia entre la estructura del código y la arquitectura documentable (15 %, la Etapa 2 asigna 55 % al diseño), la generación automática de la especificación de la API (15 %), la unificación de lenguaje con la interfaz (15 %), la madurez de bibliotecas MQTT/PostgreSQL (15 %), el tipado estático (10 %) y

el rendimiento (5 %: todas las alternativas superan con holgura los 50 mensajes por segundo requeridos).

Alternativa	**Exper.** 25 %	**Arquit.** 15 %	**OpenAPI** 15 %	**1 leng.** 15 %	**Libs** 15 %	**Tipos** 10 %	**Rend.** 5 %	**Total**
TypeScript + NestJS	4	5	5	5	5	5	4	**4,70**
TypeScript + Fastify	5	3	4	5	5	5	5	4,55
Python + FastAPI	3	3	5	2	5	4	3	3,55
Java + Spring Boot	2	5	4	1	5	5	4	3,45
Go + Echo	1	3	3	1	4	5	5	2,65

Cuadro C.2. Matriz de tecnología de backend (escala 1–5).

Se selecciona **NestJS**: sus módulos corresponden uno a uno con los módulos M1–M8 del análisis (los diagramas de la Etapa 2 describirán el código real), y con TypeScript en backend e interfaz los tipos de la trama se definen una sola vez para todo el sistema. La diferencia con Fastify (0,15) no es decisiva y migrar sería una desviación aceptable si la curva de NestJS resultara costosa. **La interfaz (frontend)** se decide en la Etapa 2, con el diseño de vistas en mano y tres restricciones ya fijadas: TypeScript, componentes de código abierto y uso en navegador móvil; se evaluarán React, Vue y Svelte con una matriz análoga.

C.5. Evaluación de la tecnología de interfaz (frontend)

El panel es el único módulo que el usuario final ve, pero **la restricción dominante de este trabajo es la ligereza**: la plataforma debe operar en hardware modesto, por debajo de las exigencias del referente (ThingsBoard recomienda 4 GB de memoria solo para sí mismo). Por ello el criterio de mayor peso es el tamaño del artefacto entregado al navegador y el consumo en el servidor que lo sirve, seguido de la coherencia con el backend (TypeScript), la disponibilidad de componentes de gráficos de series de tiempo, y la experiencia previa.

Alternativa	**Ligereza**	**TypeScript**	**Gráficos series**	**Experien.**	**Ecosist.**	**Total**
	35 %	20 %	20 %	15 %	10 %	
Svelte + uPlot	5	5	5	3	3	**4,45**
Vue + ECharts	3	5	4	4	4	3,75
React + Recharts	2	5	5	4	5	3,70
Angular (como ThingsBoard)	1	5	4	2	4	2,85

Cuadro C.3. Matriz de tecnología de interfaz (escala 1–5).

Se recomienda **Svelte con uPlot**. Svelte compila a JavaScript mínimo sin *virtual DOM*, lo que produce artefactos varias veces más livianos que React o Angular —coherente con la ligereza que diferencia a la plataforma de ThingsBoard, construido en Angular—; uPlot es una biblioteca de gráficos especializada en series de tiempo de unos pocos kilobytes, apropiada para paneles de telemetría en navegador móvil. Se mantiene TypeScript, con lo que los tipos del modelo interno se comparten entre backend e interfaz. La decisión se confirma en la Etapa 2 con el diseño de las vistas en mano; React queda como alternativa si se prioriza el ecosistema de componentes sobre la ligereza.

C.6. Presupuesto de recursos frente al referente

La preocupación por el consumo de hardware es legítima y se aborda con una estimación explícita del uso de memoria en reposo de cada componente:

Componente	Memoria en reposo	Nota
Mosquitto (broker)	∼10 MB	escrito en C
TimescaleDB (PostgreSQL)	∼150–300 MB	crece con la carga; configurable
Backend NestJS (Node)	∼80–150 MB	
ChirpStack (si hay rama LoRa)	∼50 MB	escrito en Go
Interfaz servida (estática)	∼10 MB	Svelte compilado
Total estimado en reposo	**∼0,3–0,5 GB**	frente a ≥4 GB de ThingsBoard

Cuadro C.4. Presupuesto de memoria estimado. Da origen al requerimiento RNF-17: la plataforma completa debe operar en un servidor de 2 GB de memoria, lo que se verifica empíricamente en la Etapa 4.

Esta estimación es preliminar y se somete a medición real en el banco de comparación de la Etapa 4: es una hipótesis de diseño, no una afirmación cerrada. El riesgo de que TimescaleDB crezca bajo carga se controla con políticas de retención y compresión.

C.7. Costos de alojamiento (preliminar; el detalle es OE3)

Servidor virtual arrendado: 6–20 USD/mes (2 vCPU, 4 GB). Mini-PC propio en la organización: 150–250 USD por única vez. Servidor institucional: sin costo si se dispone. Todos los componentes son de código abierto sin licencias.

Apéndice D

Camino evaluado y descartado: el protocolo Contact ID

Durante la fase inicial de esta etapa (17 de agosto al 1 de septiembre) el trabajo evaluó como eje el protocolo **Contact ID (SIA DC-05)** de la industria de alarmas, sobre transporte DC-09. Se verificó empíricamente su gramática y su suma de comprobación con una implementación propia, se evaluaron ocho alternativas mediante una matriz ponderada y se diseñó un perfil de aplicación. Ese material completo permanece en el repositorio del proyecto (`docs/05-protocolo-contactid.md` y `plataforma/herramientas/contactid.py`).

Razón del descarte (reunión con el profesor guía, 2 de septiembre de 2026): Contact ID está orientado a **eventos discretos** —alarmas que ocurren o no— mientras que el problema de este trabajo es **telemetría continua**: lecturas periódicas cada 5 a 30 minutos cuyo valor está en la serie, no en el evento. El mensaje de 16 dígitos no transporta valores medidos de forma nativa, y las extensiones necesarias para hacerlo desplazaban al protocolo fuera de su dominio de diseño.

Lo que el análisis aportó y se conserva: la restricción de que el silencio de un sensor debe detectarse activamente (los códigos de supervisión de la industria de alarmas inspiran los requerimientos RF-24 a RF-30, hoy resueltos con el *keep-alive* y el testamento de MQTT); la disciplina de evaluar alternativas con criterios ponderados y de verificar afirmaciones con código; y la separación lógica entre eventos y telemetría, que sobrevive dentro del modelo interno.

El registro de la decisión, con las tareas derivadas, está en `bitacora/2026-09-02-reunion-pivote.md`.